

System Description: A High-Fidelity Multimodal Data Collection Platform for Velocity-Based Training Research

Author Name
Department of XYZ, University of XYZ
email@university.edu

May 6, 2026

Abstract

This document presents a comprehensive system description of a custom-engineered, multi-modal data collection platform designed for PhD-level Velocity-Based Training (VBT) research. The system integrates a TDK InvenSense ICM-42688-P inertial measurement unit, sampled at a deterministic 1 kHz via interrupt-driven SPI on an ESP32 microcontroller, with an Intel RealSense D455 stereo depth camera operating at 90 fps. Hardware-level synchronisation is achieved through a discrete NPN-transistor level-shifting circuit that converts the D455's 1.8 V CMOS `FRAME_SYNC` output to 3.3 V logic, driving both the ESP32 GPIO and the IMU's internal `FSYNC` tagging engine simultaneously. Wireless data telemetry from the barbell-mounted sensor node uses ESP-NOW with 8-sample batching, yielding a 125 Hz radio duty cycle well within the protocol's capacity. On the host side, a C++17 application built with CMake manages real-time data ingestion, marker tracking, and session orchestration, while a Python-based golden-model pipeline implements attitude estimation (Madgwick, MEKF, and VQF), a 15-state Error-State Kalman Filter with lifting-specific Zero-Velocity Updates, and camera-oracle fusion strategies for sub-50 mm positional accuracy. All sessions are persisted in a schema-versioned (v4) JSON format encompassing over 120 metadata fields spanning athlete anthropometrics, training periodisation, loading provenance, and subjective quality assessments, with optional BIDS-compliant directory layouts for multi-subject archiving.

Contents

1	Introduction	5
1.1	Velocity-Based Training: Principles and Significance	5
1.2	Limitations of Existing VBT Measurement Technologies	5
1.2.1	Linear Position Transducers (LPTs)	5
1.2.2	Camera-Based Systems	6
1.2.3	Wearable Inertial Measurement Units	6
1.3	Motivation for a Custom Research Platform	6
1.4	Research Objectives	7
1.5	Document Organisation	7
2	System Architecture	7
2.1	Overview	7
2.2	Component Selection Rationale	8

2.2.1	Intel RealSense D455 Stereo Depth Camera	8
2.2.2	ESP32-WROOM-32 Microcontroller	8
2.2.3	TDK InvenSense ICM-42688-P IMU	9
2.3	Data Flow	9
2.4	Latency Budget	9
3	Hardware Design and Configuration	10
3.1	Inertial Measurement Unit: ICM-42688-P	10
3.1.1	Device Overview	10
3.1.2	Full-Scale Range Selection	10
3.1.3	Output Data Rate and Anti-Aliasing Filters	11
3.1.4	Register Map	11
3.1.5	FIFO Bypass Rationale	12
3.1.6	Scale Factors	12
3.2	Optical Tracking Sensor: Intel RealSense D455	12
3.2.1	Streaming Configuration	12
3.2.2	IR Marker Tracking Strategy	13
3.3	Barbell Sensor Node: Electrical Design	14
3.3.1	SPI Wiring	14
3.3.2	Power Management	14
3.3.3	ESP-NOW Radio Configuration	14
4	Hardware-Level Synchronisation Strategy	15
4.1	The D455 Master Pulse	15
4.2	The NPN Level-Shifting Circuit	15
4.2.1	Option A: Non-Inverting Two-Transistor Circuit (Implemented)	16
4.2.2	Bill of Materials	17
4.2.3	Grounding	17
4.3	FSYNC Signal Routing	17
4.4	IMU FSYNC Tagging Engine	17
4.5	ESP32 Interrupt Handler and Debouncing	18
4.6	Host-Side Temporal Alignment: The SyncEngine	18
4.6.1	FSYNC-Based Hardware Anchor Pairs	18
4.6.2	Tap-Test Calibration	19
4.6.3	Drift Monitoring and Auto-Rearm	19
5	Firmware Implementation	19
5.1	Barbell Node Firmware (v4.1)	19
5.1.1	Boot Sequence	19
5.1.2	Sampling Modes	20
5.1.3	SPI Burst Read	20
5.1.4	Binary Packet Format	21
5.1.5	CRC16-CCITT Integrity Check	21
5.1.6	ESP-NOW Batching	21
5.1.7	FSYNC Detection Logic	22
5.1.8	Periodic Status Reporting	22
5.2	Relay Node Firmware (v2.0)	22
5.2.1	Receive ISR and Ring Buffer	22

5.2.2	Main Loop: Drain to Serial	23
6	Host-Side Software Architecture	23
6.1	Software Architecture Overview	23
6.2	Sensor Layer	23
6.2.1	IMUReader	23
6.2.2	CameraReader	24
6.3	Session Lifecycle	24
6.3.1	Atomic Session Finalisation	25
6.3.2	Multi-Set Support	25
6.4	Data Logging	25
6.5	Real-Time Processing	26
6.5.1	Orientation Filter	26
6.5.2	Repetition Segmenter	26
6.6	Calibration System	27
6.7	Build Provenance	27
6.8	Graphical User Interface	27
7	Mathematical Data Processing Pipeline	28
7.1	Step 0: Data Loading and Visualisation	28
7.2	Step 1: Attitude Estimation and Gravity Removal	28
7.2.1	Rotation via Quaternions	28
7.2.2	Initial Orientation from Gravity Vector	29
7.2.3	Madgwick AHRS Filter	29
7.2.4	Multiplicative Extended Kalman Filter (MEKF)	29
7.2.5	Versatile Quaternion-based Filter (VQF)	29
7.3	Step 2: Velocity Integration	30
7.4	Step 3: Autonomous ZUPT Detection	30
7.5	Step 7: Camera-Oracle ZUPT Strategies	30
7.6	Step 8: 15-State Error-State Kalman Filter	30
7.6.1	State Vector	31
7.6.2	Process Model	31
7.6.3	Error-State Transition Matrix	31
7.6.4	ZUPT Observation Model	31
7.6.5	Zero Angular Rate Update (ZARU)	32
7.6.6	Zero Acceleration Update (ZAU)	32
7.6.7	Lifting-Specific Near-Rest Detector	32
7.6.8	Filter Noise Parameters	33
7.7	Repetition Segmentation in the Golden Model	33
8	Dataset Format and Metadata Schema	33
8.1	Directory Structure	33
8.2	Schema Versioning	34
8.3	Session Metadata Taxonomy: <code>SessionInfo</code>	34
8.4	Subject Metadata: <code>SubjectInfo</code>	35
8.5	Training Context: <code>SetInfo</code>	35
8.6	Equipment Provenance: <code>EquipmentInfo</code>	36
8.7	Calibration Provenance	36

8.8	Event Log (<code>events.jsonl</code>)	37
8.9	Anonymisation Pipeline	37
9	Validation and Quality Assurance	37
9.1	Pre-Session Validation: Pre-Flight Checklist	37
9.2	In-Session Monitoring	38
9.2.1	IMU Signal Quality Indicators	38
9.2.2	FSYNC Health Monitoring	38
9.2.3	Synchronisation Drift Monitoring	38
9.2.4	Per-Repetition Kinematic Plausibility	39
9.3	Post-Session Validation: Golden Model Pipeline	39
9.3.1	Camera–IMU Position Agreement	39
9.3.2	Velocity Agreement	39
9.3.3	Bland–Altman Analysis	39
9.3.4	Per-Rep ROM Accuracy	40
9.4	Synchronisation Validation: Tap Test	40
9.5	Allan Variance Characterisation	40
9.6	Diagnostic Export	40
9.7	Validation Summary and Acceptance Criteria	41

1 Introduction

1.1 Velocity-Based Training: Principles and Significance

Velocity-Based Training (VBT) is an autoregulatory strength-training methodology that prescribes and monitors training loads through the real-time measurement of barbell movement velocity, rather than relying on fixed percentages of a static One-Repetition Maximum (1RM). The theoretical foundation of VBT rests on the well-established, quasi-linear *load-velocity relationship*: as the external load on a barbell increases, the maximum concentric velocity achievable by a given athlete decreases in a predictable, approximately linear fashion for compound exercises [5]. This relationship allows practitioners to estimate the athlete’s daily 1RM from submaximal warm-up velocities, eliminating the need for periodic maximal testing—a practice that carries injury risk, induces excessive fatigue, and is unreliable for athletes in the early phases of a training block.

The kinematic variables of primary interest in VBT research are:

- **Mean Concentric Velocity (MCV)**: the average vertical velocity during the concentric (upward) phase of a lift. This is the standard metric for load prescription.
- **Peak Concentric Velocity (PCV)**: the instantaneous maximum velocity during the concentric phase. PCV is more sensitive to neuromuscular capacity but more susceptible to measurement noise.
- **Range of Motion (ROM)**: the total vertical displacement of the barbell during a complete repetition cycle.
- **Intra-Set Velocity Loss (%VL)**: the percentage decrease in MCV from the fastest repetition (typically rep 1) to the final repetition of a set.

The velocity zones corresponding to distinct strength qualities, as established in the literature, are summarised in Table 1.

Table 1: Velocity zones and corresponding strength qualities (adapted from Weakley et al. [11]).

Strength Quality	MCV Range (m/s)	Approx. %1RM
Absolute Strength	0.15–0.50	85–100%
Accelerative Strength	0.50–0.75	70–85%
Strength-Speed	0.75–1.00	55–70%
Speed-Strength	1.00–1.30	40–55%
Starting Strength	> 1.30	< 40%

A deviation of even 0.05 m/s in MCV can shift the intended training stimulus from one physiological adaptation zone to another. This places stringent accuracy requirements on VBT measurement devices—requirements that many commercial systems fail to meet under field conditions.

1.2 Limitations of Existing VBT Measurement Technologies

The commercial VBT ecosystem is populated by three dominant sensor modalities, each with significant limitations for research applications:

1.2.1 Linear Position Transducers (LPTs)

LPTs (e.g., GymAware, Tendo Unit) measure the displacement of a retractable cable tethered to the barbell. While traditionally considered the “gold standard” for VBT velocity measurement [3],

LPTs suffer from fundamental limitations. The physical tether constrains the barbell to predominantly vertical motion, altering the natural bar path during explosive multi-joint lifts such as the snatch and clean, where significant horizontal displacement is biomechanically essential. Furthermore, LPTs provide only a single degree of freedom (1-DOF, vertical displacement); they cannot capture barbell tilt, lateral drift, or rotational dynamics—kinematic features that are clinically relevant for technique analysis and injury-risk assessment.

1.2.2 Camera-Based Systems

Marker-based optoelectronic motion capture systems (Vicon, Qualisys, OptiTrack) provide the highest accuracy (< 0.1 mm positional RMS in calibrated volumes) and are the true criterion standard for biomechanical research. However, their cost (typically \$50,000–\$250,000 for a multi-camera installation), fixed laboratory footprint, and extensive calibration requirements make them impractical for routine data collection in strength-training facilities. Video-based systems using smartphone or webcam feeds (e.g., the “My Lift” application [2]) are more accessible but suffer from rolling-shutter distortion, limited frame rates (30–60 fps), and sensitivity to lighting conditions.

1.2.3 Wearable Inertial Measurement Units

Commercial IMU-based VBT devices (e.g., PUSH Band, Vmaxpro, Beast Sensor, RepOne) solve the tethering and portability problems but introduce severe algorithmic challenges. These devices operate as “black boxes,” streaming heavily filtered velocity summaries over Bluetooth Low Energy (BLE) with undocumented sampling rates, proprietary sensor-fusion algorithms, and unquantified latency. Pérez-Castilla et al. [9] reported Limits of Agreement (LoA) as wide as ± 0.11 m/s for commercial IMU devices against an LPT criterion—an error envelope that spans nearly an entire velocity zone in Table 1.

The fundamental challenge of IMU-based VBT is the double integration of accelerometer data. Velocity is obtained by integrating linear acceleration, and displacement is obtained by integrating velocity. Any uncompensated bias b_a in the accelerometer produces a position error that grows *quadratically* with time:

$$\epsilon_p(t) = \frac{1}{2} b_a t^2 \quad (1)$$

For a typical MEMS accelerometer bias of 5 mg (0.049 m/s²), this yields a positional error of 1.23 m after only 10 s—an obviously catastrophic failure for a barbell that may have moved only 0.5 m.

1.3 Motivation for a Custom Research Platform

The limitations described in Section 1.2 reveal a critical gap in the research ecosystem. To develop and validate novel IMU-based tracking algorithms—including Error-State Kalman Filters (ESKF), Versatile Quaternion-based Filters (VQF) [6], and lifting-specific Zero-Velocity Update (ZUPT) detectors [10]—researchers require access to raw, unfiltered, high-rate sensor data that is:

1. **Temporally synchronised** with a concurrent optical ground-truth signal at the *hardware* level, not via software timestamps;
2. **Fully transparent** in its processing chain, with every filtering step, calibration parameter, and algorithmic assumption documented and reproducible;
3. **Richly contextualised** with the metadata necessary for stratified analysis (athlete demographics, training phase, loading conditions, subjective fatigue, equipment provenance).

This manuscript describes the complete engineering of such a platform.

1.4 Research Objectives

The specific objectives of the data collection system are:

1. To capture raw, unfiltered 1 kHz 6-DOF IMU data at ± 16 g accelerometer and $\pm 2000^\circ/\text{s}$ gyroscope full-scale ranges, preserving the high-frequency shock transients of weightlifting movements.
2. To achieve sub-millisecond hardware synchronisation between the IMU and camera streams through a dedicated level-shifting circuit and the ICM-42688-P’s internal FSYNC tagging engine.
3. To wirelessly transmit the high-bandwidth IMU data stream from the barbell to the host PC via ESP-NOW, avoiding any physical tethering of the barbell.
4. To process all raw data through an open-source, mathematically rigorous pipeline, producing “golden model” kinematic trajectories validated against the optical ground truth.
5. To persist every session in a schema-versioned, BIDS-compatible JSON taxonomy with over 120 metadata fields, enabling fully reproducible longitudinal analysis.

1.5 Document Organisation

The remainder of this manuscript is organised as follows. Section 2 presents the high-level system architecture and the rationale for each hardware selection. Section 3 details the hardware configuration of each sensor node, including register-level IMU settings and camera parameters. Section 4 describes the hardware synchronisation strategy in depth, including the NPN level-shifting circuit, FSYNC tagging, and host-side temporal alignment. Section 5 covers the ESP32 firmware architecture for both the barbell node and the relay node. Section 6 documents the host-side C++17 application architecture. Section 7 presents the mathematical data processing pipeline, including attitude estimation, ESKF formulation, and ZUPT strategies. Section 8 formalises the dataset schema and metadata taxonomy. Section 9 describes the multi-tiered validation framework.

2 System Architecture

2.1 Overview

The data collection platform is designed around a physically distributed, wireless topology that fully decouples the barbell-mounted sensing hardware from the host workstation. This decoupling is a strict biomechanical requirement: any physical cable attached to the barbell introduces spurious restoring forces, alters the natural bar path, and compromises the ecological validity of the kinematic measurements. The system comprises four distinct functional units, depicted schematically in Figure 1.

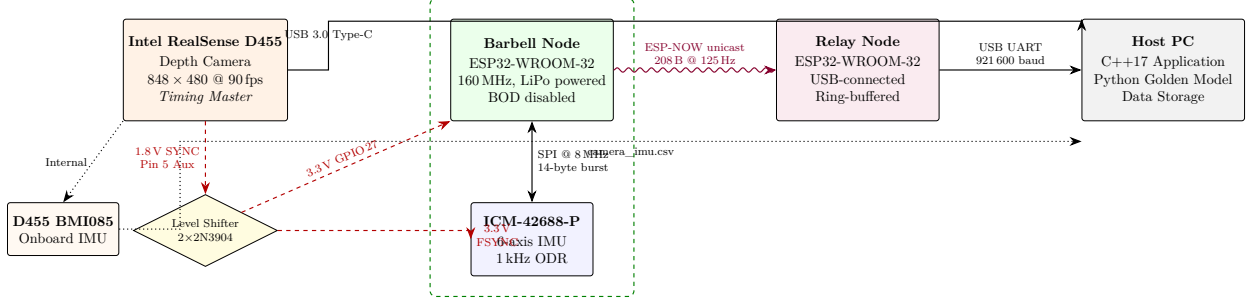


Figure 1: Complete system architecture showing data buses, wireless telemetry, and hardware synchronisation paths. Red dashed lines indicate the 1.8 V $\rightarrow$ 3.3 V level-shifted SYNC pulse. Purple wavy lines indicate wireless ESP-NOW transmission.

2.2 Component Selection Rationale

2.2.1 Intel RealSense D455 Stereo Depth Camera

The D455 was selected as the optical ground-truth sensor for the following reasons:

1. **Global shutter:** Both stereo IR imagers use global shutters, eliminating the rolling-shutter distortion that corrupts velocity estimation during fast barbell transients. At a peak velocity of 2 m/s, a rolling shutter with a 10 ms readout produces up to 20 mm of intra-frame geometric distortion.
2. **Hardware sync output:** The D455 exposes an auxiliary connector with a dedicated `FRAME_SYNC` output (Pin 5), enabling it to serve as the absolute timing master for the distributed system. This output is not available on consumer-grade depth cameras (e.g., Azure Kinect, Intel D435).
3. **Wide stereo baseline:** The 95 mm baseline between the left and right IR imagers provides reliable depth estimation up to 6 m, easily covering the operating volume of a standard power rack (2–3 m camera-to-barbell distance).
4. **Onboard IMU:** The D455 integrates a Bosch BMI085 6-axis IMU, providing accelerometer data at 250 Hz and gyroscope data at 200 Hz. While this IMU is not used for VBT kinematics (the camera body is stationary on a tripod), it provides free cross-stream synchronisation validation and tripod-shake/bump detection.
5. **Cost:** Approximately \$350 USD, three orders of magnitude cheaper than a multi-camera optoelectronic installation.

2.2.2 ESP32-WROOM-32 Microcontroller

The Espressif ESP32 was chosen over alternative microcontrollers (e.g., STM32, nRF52, RP2040) specifically for its **ESP-NOW** protocol:

- ESP-NOW operates directly on the IEEE 802.11 MAC layer, bypassing the TCP/IP and WiFi association overhead entirely. This yields sub-2 ms one-way latency with acknowledgement and automatic retry.
- Traditional BLE (Bluetooth Low Energy) is fundamentally unsuitable for 1 kHz streaming: BLE connection intervals are negotiated in multiples of 1.25 ms with typical minimums of 7.5 ms, creating severe bottlenecks. Even with BLE 5.0's extended data length, sustained throughput above 20 kB/s requires careful negotiation and is not reliable under the electromagnetic interference present in a gym environment.

- ESP-NOW supports unicast with hardware-level ACK, achieving $> 99\%$ packet delivery at maximum TX power (20 dBm).

2.2.3 TDK InvenSense ICM-42688-P IMU

The ICM-42688-P is a premium 6-axis MEMS MotionTracking device. It was selected for:

- Ultra-low gyroscope noise density: $2.8 \text{ m}^\circ/\text{s}/\sqrt{\text{Hz}}$, which is critical for accurate attitude estimation over multi-second integration windows.
- Configurable digital anti-aliasing filters: 1st, 2nd, or 3rd order, with bandwidth selectable as a fraction of the ODR.
- Internal FSYNC tagging engine: external synchronisation pulses are automatically tagged into the data payload (TEMP register LSB), eliminating the need for additional host-side interrupt handling.
- Wide full-scale ranges: $\pm 16 \text{ g}$ accelerometer and $\pm 2000^\circ/\text{s}$ gyroscope, sufficient for even the most explosive Olympic weightlifting movements.

2.3 Data Flow

The data flow through the system proceeds as follows:

1. The D455 camera generates a `FRAME_SYNC` pulse every 11.111 ms (90 Hz).
2. This pulse passes through the $2 \times 2\text{N}3904$ level shifter, producing a clean 3.3 V rising edge.
3. The shifted pulse simultaneously triggers: (a) a rising-edge ISR on ESP32 GPIO 27, which records a 64-bit τ_s timestamp; and (b) the ICM-42688-P's internal FSYNC latch, which sets the LSB of the temperature register in the next data-ready sample.
4. At 1 kHz, the ICM-42688-P asserts INT1 upon each new sample. The ESP32 ISR sets a boolean flag; the main loop then executes a 14-byte SPI burst read, packs the data into a 26-byte CRC-protected packet, and appends it to a batching buffer.
5. Every 8 samples (8 ms), the 208-byte batch is transmitted via ESP-NOW unicast to the relay node.
6. The relay node receives the ESP-NOW packet in an ISR, appends it to an 8 KB ring buffer, and the main loop drains the buffer to the USB UART at 921 600 baud.
7. The host PC ingests both the serial IMU stream and the USB 3.0 camera stream. The `SyncEngine` class maintains a live affine mapping between ESP timestamps and camera hardware timestamps using the FSYNC tag events.

2.4 Latency Budget

Table 2 summarises the end-to-end latency from physical sensor event to host-side availability.

Table 2: End-to-end latency budget from sensor event to host availability.

Stage	Source	Latency
IMU ADC conversion	ICM-42688-P internal	$< 100 \text{ }\mu\text{s}$
SPI burst read (14 B)	ESP32 @ 8 MHz	$\sim 14 \text{ }\mu\text{s}$
Packet assembly + CRC	ESP32 CPU	$\sim 5 \text{ }\mu\text{s}$
Batch accumulation (8 pkt)	ESP32 RAM	8 ms (worst case)
ESP-NOW TX + ACK	802.11 MAC	$\sim 2 \text{ ms}$
Relay ring buffer drain	Serial @ 921 600 baud	$\sim 2.3 \text{ ms}$
USB/OS scheduling jitter	Host OS	$\sim 1 \text{ ms}$
Total (worst case)		$\sim 13.4 \text{ ms}$

Critically, the latency between sensor event and host availability is *irrelevant* for post-processing accuracy, because every sample carries its own 64-bit ESP32 hardware timestamp (`esp_timer_get_time()`), captured at the moment of the SPI read—before any batching or transmission. The latency budget matters only for the real-time GUI feedback loop (operator view, live velocity plot), where 13 ms is imperceptible.

3 Hardware Design and Configuration

This section details the register-level configuration of each sensor, the electrical interconnections, and the design rationale for every parameter choice.

3.1 Inertial Measurement Unit: ICM-42688-P

3.1.1 Device Overview

The TDK InvenSense ICM-42688-P is a 6-axis MEMS MotionTracking device integrating a 3-axis accelerometer and a 3-axis gyroscope on a single silicon die. Key performance specifications relevant to the VBT application are summarised in Table 3.

Table 3: ICM-42688-P key performance specifications (from datasheet DS-000347 Rev. 1.7).

Parameter	Accelerometer	Gyroscope
Full-Scale Range	$\pm 2/4/8/16 \text{ g}$	$\pm 15.625\text{--}\pm 2000^\circ/\text{s}$
Output Data Rate (ODR)	1.5625–32 kHz	12.5–32 kHz
Noise Density	$70 \text{ }\mu\text{g}/\sqrt{\text{Hz}}$	$2.8 \text{ m}^\circ/\text{s}/\sqrt{\text{Hz}}$
Non-Linearity	$\pm 0.1 \text{ \% FS}$	$\pm 0.05 \text{ \% FS}$
Cross-Axis Sensitivity	$\pm 1 \text{ \%}$	$\pm 1 \text{ \%}$
Zero-g Offset (typical)	$\pm 20 \text{ mg}$	$\pm 1^\circ/\text{s}$
Digital Interface	SPI (up to 24 MHz) / I ² C (1 MHz)	

3.1.2 Full-Scale Range Selection

The accelerometer full-scale range is set to the maximum $\pm 16 \text{ g}$. This choice is driven by the extreme shock environment of weightlifting. During a power clean, the barbell experiences peak accelerations exceeding 8 g at the “catch” phase, and when the barbell is dropped onto the platform (as is

standard practice in Olympic weightlifting), transient shocks can exceed 15 g. If these transients clip the ADC, the affected samples are irrecoverably corrupted—and because they occur precisely at the biomechanically most interesting moments, no amount of post-hoc interpolation can recover the lost information.

The gyroscope full-scale range is set to $\pm 2000^\circ/\text{s}$. During explosive lifts, the barbell can undergo angular rates exceeding $500^\circ/\text{s}$ during the turnover phase of a snatch. The $\pm 2000^\circ/\text{s}$ range provides a $4\times$ safety margin against clipping, at the cost of a proportional increase in quantisation noise (resolution per LSB). At 16-bit ADC resolution, the gyroscope LSB is:

$$\text{LSB}_{\text{gyro}} = \frac{2 \times 2000}{2^{16}} = 0.061^\circ/\text{s}/\text{LSB} \quad (2)$$

This resolution is more than sufficient for the angular dynamics of barbell movements, which typically exhibit angular rates above $5^\circ/\text{s}$ during active phases.

3.1.3 Output Data Rate and Anti-Aliasing Filters

The ODR is set to 1 kHz for both the accelerometer and gyroscope. This rate satisfies the Nyquist criterion for the highest-frequency kinematic content of interest (barbell vibration modes at 30–50 Hz for a standard 20 kg Olympic barbell [8]) with a generous oversampling factor of $> 10\times$.

The ICM-42688-P provides configurable digital anti-aliasing filters with selectable order (1st, 2nd, or 3rd) and bandwidth (expressed as a fraction of the ODR). We select the **3rd-order** filter with a bandwidth of **ODR/4 = 250 Hz**. This configuration provides a sharp roll-off above 250 Hz, attenuating aliased high-frequency content from the 32 kHz internal sampling clock, while preserving the full kinematic bandwidth of human movement (0–10 Hz for gross kinematics, up to 50 Hz for barbell vibration modes).

3.1.4 Register Map

Table 4 provides the exhaustive register configuration applied during initialisation.

Table 4: Complete ICM-42688-P register configuration (Bank 0 unless noted).

Register	Address	Value	Description
DEVICE_CONFIG	0x11	0x01	Soft reset (1 ms recovery)
GYRO_CONFIG0	0x4F	0x06	FS: $\pm 2000^\circ/\text{s}$, ODR: 1 kHz
ACCEL_CONFIG0	0x50	0x06	FS: $\pm 16\text{ g}$, ODR: 1 kHz
GYRO_CONFIG1	0x51	0x08	Filter order: 3rd ($0x02 \ll 2$)
ACCEL_CONFIG1	0x53	0x08	Filter order: 3rd ($0x02 \ll 2$)
GYRO_ACCEL_CFG0	0x52	0x11	BW = ODR/4 (both axes)
FIFO_CONFIG	0x16	0x00	FIFO bypassed (direct read)
INT_CONFIG	0x14	0x03	INT1: push-pull, active HIGH, pulsed
INT_CONFIG1	0x64	0x00	Async reset (clear on read)
INT_SOURCE0	0x65	0x08	Data-ready $\rightarrow$ INT1
FSYNC_CONFIG	0x62	0x12	Tag TEMP LSB; clear-on-read
TMST_CONFIG	0x54	0x1B	FSYNC_EN + TMST_EN + 1 μs
PWR_MGMT0	0x4E	0x0F	Accel + Gyro Low-Noise mode
<i>Bank 1</i>			
INTF_CONFIG5	0x7B	0x02	Pin 9 function = FSYNC input

3.1.5 FIFO Bypass Rationale

The ICM-42688-P includes a 2 KB internal FIFO for batch reads. We deliberately bypass the FIFO (FIFO_CONFIG = 0x00) for two reasons:

1. **Latency:** The FIFO introduces variable read latency (up to the FIFO watermark threshold), which is unacceptable for real-time GUI feedback.
2. **FSYNC tagging:** When the FIFO is active, the FSYNC tag is embedded in the FIFO packet header rather than the sensor register, complicating the parsing logic. Direct register reads guarantee a deterministic, flat packet format.

3.1.6 Scale Factors

The host-side software applies the following conversions from raw 16-bit integers to physical units:

$$a [\text{g}] = \text{raw}_{16} \times \frac{16.0}{32768.0} \quad (3)$$

$$\omega [^\circ/\text{s}] = \text{raw}_{16} \times \frac{2000.0}{32768.0} \quad (4)$$

$$T [^\circ\text{C}] = \text{raw}_{16}/132.48 + 25.0 \quad (5)$$

3.2 Optical Tracking Sensor: Intel RealSense D455

3.2.1 Streaming Configuration

Table 5 presents the complete camera configuration as applied via the librealsense2 API.

Table 5: D455 streaming configuration.

Parameter	Value	Rationale
IR Resolution	848×480	Optimal balance of spatial resolution and USB bandwidth.
Frame Rate	90 fps	Maximum supported; captures 11.1 ms transients.
Exposure	1758 μ s	Fast enough to limit motion blur to < 3.5 mm at 2 m/s.
Gain	164	Compensates for the short exposure under indoor lighting.
IR Emitter	OFF	Prevents the structured-light dot pattern from confusing the blob detector.
Depth Stream	Enabled	Required for 3D deprojection of detected markers.
RGB Stream	Disabled	The $848 \times 480 \times 3$ B @ 30 fps (~ 36 MB/s) payload saturated the USB 3.0 bus, causing intermittent frame drops.
Hardware Sync Mode	1 (Master)	Configures Pin 5 to output the 1.8 V <code>FRAME_SYNC</code> pulse.
<i>Onboard IMU (Bosch BMI085)</i>		
Accelerometer Rate	250 Hz	Used for tripod-shake detection; not for VBT kinematics.
Gyroscope Rate	200 Hz	Logged to <code>camera_imu.csv</code> for cross-stream sync validation.

3.2.2 IR Marker Tracking Strategy

Rather than using the D455’s active structured-light projector, our tracking pipeline relies on **passive retroreflective IR markers** affixed to the barbell collar. With the emitter disabled, these markers appear as high-contrast bright blobs against the dark background of the gym in the left IR image. The marker detection pipeline performs:

1. Binary thresholding at a configurable threshold (currently 169).
2. Horizontal ROI masking (15%–85% of frame width) to reject door frames, windows, and spectators.
3. Morphological opening to remove single-pixel noise.
4. Connected-component (blob) analysis with area filtering (5–2000 pixels).
5. Circularity scoring and Signal-to-Noise Ratio (SNR) computation.
6. Selection of the best-scoring blob as the tracked marker.
7. 3D deprojection using the aligned depth frame and the camera intrinsics via `rs2_deproject_pixel_to_point`.
8. Robust depth estimation using a median filter over a $(2w + 1)^2$ patch (default $w = 5$) to suppress single-pixel depth noise.

The marker tracker (**MarkerTracker**) maintains a 10-frame detection history for temporal consistency. When the marker is briefly lost (e.g., occluded by the lifter’s body at the bottom of a squat), the tracker uses the last valid detection for prediction rather than coasting with a Kalman

filter, avoiding the introduction of model-dependent artefacts into the ground-truth signal.

3.3 Barbell Sensor Node: Electrical Design

3.3.1 SPI Wiring

The ICM-42688-P communicates with the ESP32 via a 4-wire SPI bus at 8 MHz. The pin mapping is:

Table 6: SPI wiring between ESP32 and ICM-42688-P.

Function	ESP32 GPIO	ICM-42688-P Pin
SCK (Clock)	GPIO 18	Pin 7 (SCLK)
MISO (SDO)	GPIO 19	Pin 5 (SDO)
MOSI (SDI)	GPIO 23	Pin 6 (SDI)
CS (Chip Select)	GPIO 5	Pin 8 (nCS)
INT1 (Data Ready)	GPIO 26	Pin 4 (INT1)
FSYNC	Direct from level shifter output (see Section 4)	

3.3.2 Power Management

The barbell node is powered by a 3.7 V, 500 mA h lithium-polymer cell housed in the 3D-printed collar mount. To prevent nuisance resets during the voltage sags caused by simultaneous SPI burst reads, ESP-NOW radio transmissions, and mechanical shocks from barbell drops, two power-management measures are implemented:

1. **Brown-Out Detector (BOD) disabled:** The ESP32’s brownout threshold (~ 2.43 V) is too aggressive for marginal battery conditions. Disabling the BOD via `WRITE_PERI_REG(RTC_CNTL_BROWN_OUT_R, 0)` allows the system to survive brief sags without resetting. The risk of flash corruption is bounded because flash writes occur only during firmware upload, never during data collection.
2. **CPU frequency reduction:** The clock is reduced from 240 MHz to 160 MHz via `setCpuFrequencyMhz(160)`. Profiling shows that the complete SPI read, packet assembly, and CRC computation require $< 10 \mu\text{s}$ per 1 ms cycle—a $< 1\%$ CPU utilisation. The remaining 99% is idle, and the frequency reduction saves approximately 30% average current draw, extending battery life to > 12 hours of continuous operation.

3.3.3 ESP-NOW Radio Configuration

The ESP-NOW transmitter is configured for maximum reliability:

- **Unicast addressing:** The relay node’s MAC address (`68:25:DD:32:78:54`) is hardcoded as a peer. Unicast provides hardware-level ACK and automatic retry, unlike broadcast which is rate-limited to ~ 100 Hz on ESP32 and drops $> 99\%$ of packets at the IMU’s 988 Hz native rate.
- **Maximum TX power:** `esp_wifi_set_max_tx_power(80)` sets the output to 20 dBm (100 mW), overcoming the Faraday-cage shielding effect of the metal barbell and potential body occlusion during lifts.
- **Station mode:** The WiFi radio is set to `WIFI_STA` mode with no active connection, ensuring the radio hardware is available exclusively for ESP-NOW.

4 Hardware-Level Synchronisation Strategy

Temporal synchronisation is the most critical engineering challenge in multimodal kinematic data collection. A synchronisation error of just 10 ms between the IMU and camera streams—less than one camera frame period—introduces a velocity error proportional to the barbell’s jerk (rate of change of acceleration). During the “sticking point” of a heavy squat, where acceleration can change by 5 m/s^2 in 10 ms, a 10 ms timing misalignment produces a systematic velocity bias of 0.05 m/s —equal to the entire width of a velocity training zone.

Software-based synchronisation approaches (NTP, PTP, or host-side timestamp matching) are fundamentally inadequate for this application. They suffer from unpredictable OS scheduling delays (typically 1–15 ms on Linux), USB bus arbitration jitter, and thermal drift between independent crystal oscillators. Our system therefore implements a *direct hardware synchronisation topology* that eliminates all software-timing dependencies.

4.1 The D455 Master Pulse

The D455 camera is configured as the system timing master (`hw_sync_mode = 1`). At the precise start of exposure for every depth frame, the camera drives a physical `FRAME_SYNC` pulse on Pin 5 of its 9-pin auxiliary connector. At 90 fps, this pulse fires exactly every 11.111 ms with sub-microsecond jitter (governed by the D455’s internal PLL).

This output signal is a 1.8 V CMOS logic level.

4.2 The NPN Level-Shifting Circuit

The ESP32 and the ICM-42688-P both operate at 3.3 V logic levels. The ESP32’s nominal input-high voltage threshold is $V_{IH} \approx 0.75 \times V_{DD} = 2.475 \text{ V}$, which exceeds the 1.8 V camera output. While empirical testing showed that the physical silicon threshold often sits lower (around 1.4–1.6 V), relying on out-of-spec behaviour is unacceptable for a research platform that must produce reliable data across temperature variations and unit-to-unit silicon variability.

We therefore designed and built a discrete NPN transistor level-shifting circuit. Two circuit topologies were evaluated, and the recommended Option A (non-inverting, two-transistor) was implemented. The complete schematic and bill of materials are shown in Figure 2.

D455 (1.8V SYNC) to ESP32 / IMU FSYNC (3.3V) Level Shifter Options

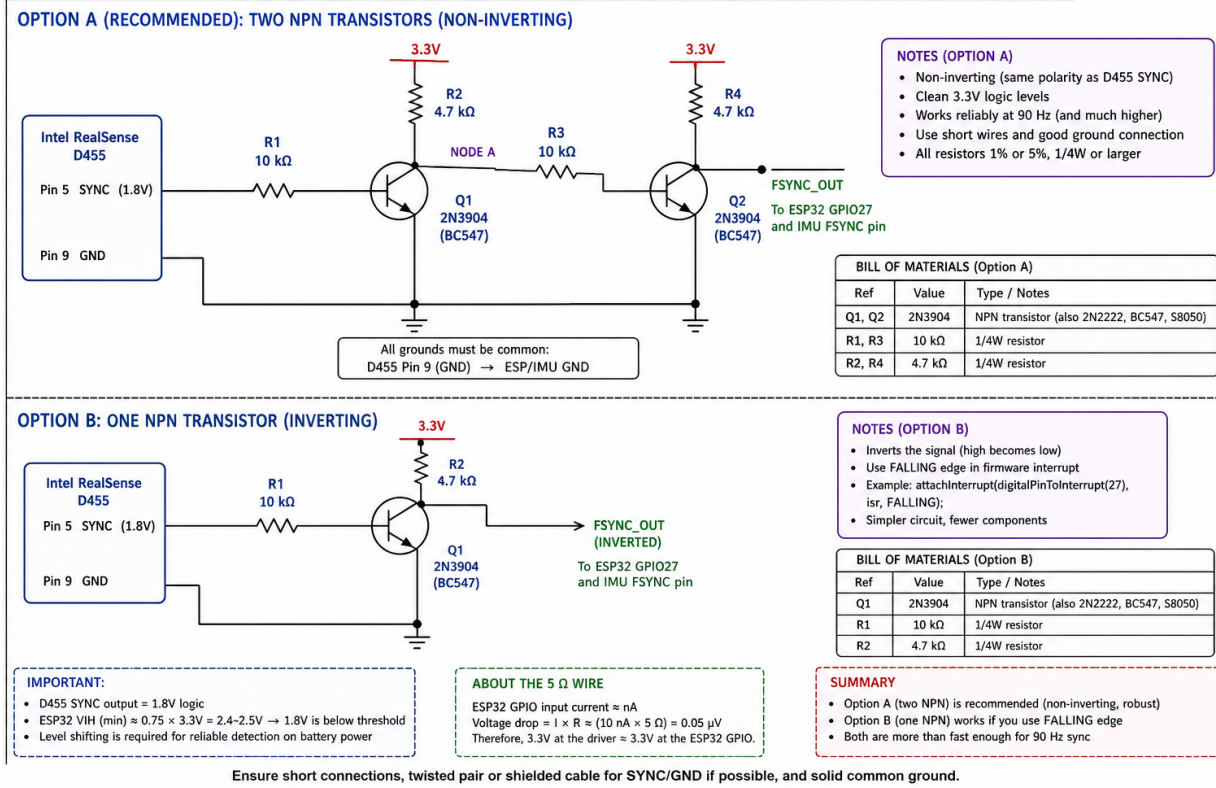


Figure 2: Complete level-shifter schematic showing both evaluated topologies. Option A (top, recommended) uses two 2N3904 NPN transistors in a non-inverting configuration, preserving the SYNC polarity. Option B (bottom) uses a single transistor but inverts the signal, requiring a FALLING-edge interrupt configuration in firmware.

4.2.1 Option A: Non-Inverting Two-Transistor Circuit (Implemented)

The recommended circuit uses two 2N3904 (or equivalent BC547/S8050) NPN transistors:

- **Q1 (First Stage):** The D455 SYNC output (Pin 5, 1.8 V) drives the base of Q1 through a 10 k current-limiting resistor (R1). Q1's collector is pulled up to 3.3 V via a 4.7 k resistor (R2). When SYNC goes HIGH, Q1 turns on, pulling NODE A LOW. This constitutes the first inversion.
- **Q2 (Second Stage):** NODE A drives the base of Q2 through a 10 k resistor (R3). Q2's collector is pulled up to 3.3 V via a 4.7 k resistor (R4). When NODE A goes LOW (Q1 on), Q2 turns off, and the output (FSYNC_OUT) is pulled HIGH to 3.3 V. This constitutes the second inversion, restoring the original polarity.

The result is a clean 3.3 V output that faithfully reproduces the input polarity with a propagation delay $< 100\text{ ns}$ —negligible relative to the 11.1 ms frame period.

4.2.2 Bill of Materials

Table 7: Bill of Materials for the level-shifting circuit (Option A).

Ref	Value	Type	Notes
Q1, Q2	2N3904	NPN transistor	Also BC547, 2N2222, S8050
R1, R3	10 k	1/4 W resistor	1% or 5% tolerance
R2, R4	4.7 k	1/4 W resistor	1% or 5% tolerance

4.2.3 Grounding

All ground planes must be common: D455 Aux Pin 9 (GND) → ESP32 GND → ICM-42688-P GND. Short connections, twisted-pair or shielded cable for the SYNC/GND pair, and solid common grounding are essential for reliable edge detection at 90 Hz.

4.3 FSYNC Signal Routing

The level-shifted 3.3 V FSYNC_OUT signal is routed simultaneously to two destinations:

1. **ESP32 GPIO 27:** Configured as a rising-edge interrupt input.
2. **ICM-42688-P Pin 9 (FSYNC):** Configured via Bank 1 register INTF_CONFIG5 (0x7B) to accept the FSYNC function (value 0x02, setting bits[2:1] = 01).

Both chips therefore receive the identical physical edge at the same instant, guaranteeing zero inter-chip synchronisation skew.

4.4 IMU FSYNC Tagging Engine

When the 3.3 V edge arrives at the ICM-42688-P FSYNC pin, the IMU’s internal tagging engine records the event directly into the sensor data payload. The tagging is configured via FSYNC_CONFIG (address 0x62):

- **Bits [6:4] = 001:** Selects the Least Significant Bit of TEMP_DATA0 as the tag target. When a FSYNC event is detected, this bit is forced to 1 in the next data-ready sample.
- **Bit [1] = 1 (FSYNC_UI_FLAG_CLEAR_SEL):** The tag remains latched until the host performs a SPI read of the tagged register. This is critical because the camera runs at 90 Hz while the IMU is sampled at 1 kHz; without latching, the transient tag would be overwritten by the next internal UI register update before the ESP32 has a chance to read it.
- **Bit [0] = 0 (FSYNC_POLARITY):** Configures the FSYNC engine to trigger on the rising edge, matching the non-inverting level-shifter output polarity.

The complete register write is: `writeReg(0x62, 0x12)`.

Additionally, the internal timestamp counter is enabled via TMST_CONFIG (0x54) with value 0x1B, enabling:

- **TMST_TO_REGS_EN:** Latch the 16-bit internal timestamp counter at the FSYNC edge.
- **TMST_RESOL = 1:** 1 μ s timestamp resolution.
- **TMST_FSYNC_EN:** Enable FSYNC-triggered timestamp capture.
- **TMST_EN:** Enable the timestamp counter.

The captured 16-bit timestamp can be read from TMST_FSYNCH/L (address 0x2B–0x2C), providing the IMU-clock time of the FSYNC edge with 1 μ s resolution.

4.5 ESP32 Interrupt Handler and Debouncing

On the ESP32 side, GPIO 27 is configured with an interrupt on the **RISING** edge. Even with the level-shifted signal, the rising edge may exhibit brief ringing as the transistor switches, potentially causing multiple spurious ISR firings within a few microseconds. Since genuine camera frames are separated by ≥ 10 ms, a strict temporal debounce window of 5 ms is implemented directly in the ISR:

```
1 static void IRAM_ATTR isr_cam_trigger() {
2     int64_t now = esp_timer_get_time();
3     if ((now - g_last_cam_trig_us) < 5000) return; // 5 ms debounce
4     g_cam_trig_count++;
5     g_last_cam_trig_us = now;
6 }
```

Listing 1: Camera trigger ISR with debounce logic.

This handler provides two independent pieces of synchronisation information:

1. The **count** of camera trigger events (`g_cam_trig_count`), which should match the FSYNC hit count from the IMU’s temperature LSB tagging.
2. The **ESP32 timestamp** of each trigger event (`g_last_cam_trig_us`), which can be correlated with the camera’s hardware timestamp for the corresponding frame.

The 5-second status report periodically prints both the camera trigger rate and the FSYNC hit count, allowing the operator to verify in real-time that both chips are detecting the same edges.

4.6 Host-Side Temporal Alignment: The SyncEngine

On the host PC, the **SyncEngine** class maintains a unified timeline by building a live affine mapping between the three independent clock domains:

1. **ESP32 clock:** 64-bit microsecond counter (`esp_timer_get_time()`).
2. **D455 hardware clock:** Reported as Unix-epoch seconds via the `RS2_FRAME_METADATA_SENSOR_TIMESTAMP` field with `GLOBAL_TIME` domain.
3. **Host monotonic clock:** `std::chrono::steady_clock`, used as the common reference.

4.6.1 FSYNC-Based Hardware Anchor Pairs

Every IMU sample whose temperature LSB is set (`fsync_tagged = true`) was captured at the exact same physical instant as a camera frame trigger. The **SyncEngine** registers these events from both streams:

```
void register_imu_fsync_event(uint64_t esp_us, double host_s);
void register_camera_frame(double cam_hw_s, double host_s);
```

Pairing is performed by nearest-neighbour matching on the host timestamp. From the accumulated pairs $(t_{\text{esp},i}, t_{\text{cam},i})$, the engine fits a linear model:

$$t_{\text{cam}} = a \cdot t_{\text{esp}} + b \quad (6)$$

where $a \approx 10^{-6}$ (seconds per microsecond, i.e., unity scaling) and b captures the fixed offset. This affine fit is refreshed continuously as new pairs arrive, with a sliding window of up to 256 pairs, making the system self-calibrating and robust to slow thermal drift.

4.6.2 Tap-Test Calibration

Before each session, a mandatory “tap test” is performed. The operator sharply taps the IMU sensor against a rigid surface near the camera. This produces a massive, instantaneous acceleration spike ($> 2\text{ g}$) in the IMU data stream and a simultaneous visual displacement in the camera depth frame. The `SyncEngine` cross-correlates the two signals to extract:

- **Temporal offset** (μs): the systematic delay between the two clock domains.
- **Clock drift** (ppm): the rate of divergence between the ESP32 crystal and the D455 ASIC clock.
- **Correlation quality** (0–1): confidence in the computed alignment.

4.6.3 Drift Monitoring and Auto-Rearm

During recording, the `SyncEngine` continuously monitors clock drift. If drift exceeds 100 ppm for a sustained period of 5 s, the `rearm_required()` flag is set, triggering a red banner in the GUI and an audible warning tone. The operator must then perform a fresh tap test before recording can resume. These thresholds are configurable via `vbt_config.json`:

Table 8: Synchronisation quality thresholds.

Parameter	Value	Description
<code>max_drift_ppm</code>	50 ppm	Data flagged as suspect if exceeded.
<code>auto_rearm_drift_ppm</code>	100 ppm	Forces mandatory re-calibration.
<code>auto_rearm_sustain_s</code>	5 s	Sustained duration before rearm triggers.
<code>tap_threshold_g</code>	2 g	Minimum acceleration spike for tap detection.
<code>tap_test_window_ms</code>	5000 ms	Duration of the tap-test capture window.

5 Firmware Implementation

This section describes the embedded firmware running on both ESP32 microcontrollers: the barbell-mounted sensor node and the USB-connected relay node. Both are implemented in C++ using the Arduino core for ESP-IDF.

5.1 Barbell Node Firmware (v4.1)

5.1.1 Boot Sequence

The barbell node executes the following deterministic initialisation sequence:

1. **Power hardening:** Disable the Brown-Out Detector and reduce CPU frequency to 160 MHz (see Section 3).
2. **Serial initialisation:** Open the USB UART at 921 600 baud for diagnostic output (prefixed with #).
3. **SPI bus initialisation:** Configure the default VSPI bus with `SPI.begin(18, 19, 23, 5)`.

4. **IMU initialisation:** Execute the register configuration sequence in Table 4, verify WHO_AM_I = 0x47, and wait 50 ms for gyroscope startup.
5. **Camera SYNC input:** Attach the GPIO 27 rising-edge interrupt (see Listing 1).
6. **ESP-NOW initialisation:** Set WiFi to STA mode, configure maximum TX power, register the relay node as a unicast peer.
7. **INT1 probe:** Attempt to detect data-ready pulses on GPIO 26 over 50 ms. If detected, attach the interrupt-driven sampling mode; otherwise, fall back to software polling.

5.1.2 Sampling Modes

The firmware supports two mutually exclusive sampling modes:

Interrupt-Driven Mode (Primary): When INT1 is successfully detected during boot, the ESP32 attaches a RISING-edge interrupt:

```
attachInterrupt(digitalPinToInterrupt(PIN_INT1), isr_data_ready, RISING);
```

The ISR sets a volatile boolean flag `g_data_ready = true`. The main loop polls this flag and, when set, immediately executes the SPI burst read. This mode achieves sub-1 μ s sampling jitter, limited only by the ISR latency of the Xtensa LX6 core.

Polling Fallback Mode: If the INT1 wire is severed or the ISR probe fails, the firmware falls back to a software-timed polling loop. A target timestamp `g_next_sample_us` is maintained, initialised from `micros()` and advanced by 1000 μ s on each iteration. If the ESP32 falls behind by more than 5000 μ s (due to a WiFi task priority inversion, for example), the target is reset to prevent a burst of catch-up reads.

5.1.3 SPI Burst Read

On each data-ready event, the ESP32 captures a 64-bit microsecond timestamp via `esp_timer_get_time()` and then initiates a continuous 14-byte SPI burst read starting at `TEMP_DATA1` (0x1D):

Table 9: SPI burst read byte layout (14 bytes from address 0x1D).

Byte Offset	Register	Content	Endianness
0–1	TEMP_DATA1/0	Temperature	Big-endian
2–3	ACCEL_X_H/L	Accel X	Big-endian
4–5	ACCEL_Y_H/L	Accel Y	Big-endian
6–7	ACCEL_Z_H/L	Accel Z	Big-endian
8–9	GYRO_X_H/L	Gyro X	Big-endian
10–11	GYRO_Y_H/L	Gyro Y	Big-endian
12–13	GYRO_Z_H/L	Gyro Z	Big-endian

The sensor registers are big-endian (MSB first); the firmware reconstructs each 16-bit signed integer from the raw bytes:

```
int16_t ax = (int16_t)((uint16_t)raw[2] << 8 | raw[3]);
```

5.1.4 Binary Packet Format

The firmware manually assembles a 26-byte little-endian binary packet for transmission. Manual byte packing via explicit `memcpy` operations is used instead of C++ struct serialisation to avoid compiler-dependent alignment padding.

Table 10: 26-byte binary packet structure (little-endian).

Offset	Field	Type	Description
0–1	Sync Word	uint16_t	Fixed value 0x55AA for frame detection.
2–9	Timestamp	uint64_t	ESP32 ts counter (<code>esp_timer_get_time</code>).
10–11	Accel X	int16_t	Raw accelerometer X.
12–13	Accel Y	int16_t	Raw accelerometer Y.
14–15	Accel Z	int16_t	Raw accelerometer Z.
16–17	Gyro X	int16_t	Raw gyroscope X.
18–19	Gyro Y	int16_t	Raw gyroscope Y.
20–21	Gyro Z	int16_t	Raw gyroscope Z.
22–23	Temperature	int16_t	Raw temp. LSB = FSYNC tag.
24–25	CRC16	uint16_t	CRC16-CCITT over bytes 0–23.

The FSYNC tag is embedded in the temperature value: on the host, `(temp_raw & 0x01) == 1` indicates that the camera trigger pulse was latched during this sample’s acquisition period.

5.1.5 CRC16-CCITT Integrity Check

A CRC16-CCITT checksum (polynomial 0x1021, initial value 0xFFFF) is computed over the first 24 bytes and appended as bytes 24–25. The host PC runs the identical algorithm; if the checksum diverges, the packet is discarded and the gap is flagged in the statistics.

```

1  static uint16_t crc16_ccitt(const uint8_t* data, size_t length) {
2      uint16_t crc = 0xFFFF;
3      for (size_t i = 0; i < length; i++) {
4          crc ^= (uint16_t)data[i] << 8;
5          for (int j = 0; j < 8; j++) {
6              if (crc & 0x8000) crc = (crc << 1) ^ 0x1021;
7              else             crc <<= 1;
8          }
9      }
10     return crc;
11 }
```

Listing 2: CRC16-CCITT implementation (identical on ESP32 and host).

5.1.6 ESP-NOW Batching

Triggering the ESP32’s radio hardware at 1 kHz causes the RTOS scheduler to thrash and the packet drop rate to exceed 99%. The firmware therefore batches $N = 8$ consecutive 26-byte packets

into a single 208-byte buffer before invoking `esp_now_send()`. This reduces the radio duty cycle from 1000 Hz to 125 Hz, well within the ESP-NOW protocol’s demonstrated reliable throughput capacity of ~ 200 Hz in unicast mode.

The batch size of 8 was selected to keep the total payload (208 bytes) below the ESP-NOW maximum frame size of 250 bytes, while maximising the number of samples per radio transaction.

5.1.7 FSYNC Detection Logic

The firmware implements two independent FSYNC detection paths for diagnostic cross-validation:

1. **Path A (TEMP LSB):** After every SPI read, the firmware checks `raw[1] & 0x01`. If set, the FSYNC counter `g_fsync_hits` is incremented, and the 16-bit FSYNC timestamp is read from `TMST_FSYNCH (0x2B)`.
2. **Path B (INT_STATUS2 polling):** Every 50th sample (~ 20 Hz), the firmware reads `INT_STATUS2 (0x37)` and checks bit 6 (`UI_FSYNC_INT`). This register-based approach is independent of the TEMP LSB tag and provides a second confirmation channel.

During operation, the 5-second status report prints both counters, allowing the operator to verify that the level shifter, IMU FSYNC engine, and ESP32 GPIO interrupt are all functioning correctly.

5.1.8 Periodic Status Reporting

Every 5 s, the firmware emits an ASCII diagnostic string (prefixed with #) both over USB serial and via ESP-NOW to the relay. This report includes:

- IMU sampling rate and total sample count.
- Camera trigger rate and cumulative trigger count.
- FSYNC hit count and latest FSYNC timestamp.
- Raw `TEMP_DATA0` and `INT_STATUS2` register values.
- ESP-NOW packet counts (sent, failed, status).

5.2 Relay Node Firmware (v2.0)

The relay node serves as a transparent ESP-NOW-to-USB bridge. Its architecture is deliberately minimal to maximise throughput:

5.2.1 Receive ISR and Ring Buffer

The ESP-NOW receive callback `on_recv()` is invoked from the WiFi task context. Performing a blocking `Serial.write()` inside this callback would stall the WiFi task and cause ESP-NOW to drop subsequent packets. Instead, the callback appends the received bytes to an 8 KB circular ring buffer:

```

1  static void on_recv(const uint8_t* mac, const uint8_t* data, int len) {
2      g_recv_pkts++;
3      g_recv_bytes += len;
4      size_t free_space = RING_SIZE - ring_used() - 1;
5      if ((size_t)len > free_space) { g_overruns++; return; }
6      size_t w = g_ring_w;
7      for (int i = 0; i < len; i++) {
8          g_ring[w] = data[i];
9          w = (w + 1) % RING_SIZE;
10     }
11     g_ring_w = w;

```

Listing 3: Relay node receive callback with ring buffer.

5.2.2 Main Loop: Drain to Serial

The main `loop()` continuously drains the ring buffer to `Serial.write()` in 512-byte blocks. At 921 600 baud with 10 bits per byte (8N1 + start/stop), the theoretical maximum throughput is 92.16 kB/s. The IMU stream at $1 \text{ kHz} \times 26 \text{ bytes} = 26 \text{ kB/s}$ consumes only $\sim 28\%$ of this capacity, providing comfortable headroom.

The 8 KB ring buffer can absorb approximately 40 batched ESP-NOW packets (320 ms of data), providing sufficient slack for brief `Serial.write()` stalls caused by USB flow control.

6 Host-Side Software Architecture

The host-side application is a C++17 real-time data acquisition, visualisation, and session management platform. It is built with CMake 3.20+ and depends on: `librealsense2`, OpenCV 4.0+, Eigen3, `nlohmann/json`, `spdlog`, Dear ImGui 1.90.4 with ImPlot 0.16, GLFW 3.3, and OpenGL. The application version at the time of writing is v1.1.0.

6.1 Software Architecture Overview

The application follows a layered architecture with clear separation of concerns:

Table 11: Host application software layers and key classes.

Layer	Key Classes	Responsibility
<i>Sensors</i>	IMUReader, CameraReader, MarkerTracker	Hardware abstraction; streaming.
<i>Core</i>	Session, SyncEngine, DataLogger, CalibrationManager, EventLog	Session lifecycle, clock, data persistence, calibration.
<i>Processing</i>	OrientationFilter, RepSegmenter, Validator, Autoregulation	Real-time kinematics, plausibility checks.
<i>GUI</i>	MainWindow, SessionPanel, PreflightPanel, OperatorView, etc.	ImGui-based multi-panel interface.
<i>Annotation</i>	AnnotationStudio, TimelinePanel, VideoPanel, RepTablePanel	Post-session NLE-style editing and validation.
<i>Utils</i>	SerialPort, AudioCue, Notifications, RingBuffer, SubjectRegistry	Cross-platform I/O, toast system.

6.2 Sensor Layer

6.2.1 IMUReader

The `IMUReader` class manages a dedicated background thread that continuously reads from the USB serial port at 921 600 baud. Incoming bytes are appended to a 4 KB ring buffer. The parser scans for the sync word (0x55AA), extracts 26-byte packets, validates the CRC16-CCITT checksum, and converts raw integers to physical units using the scale factors in Equation (3)–Equation (5).

Quality metrics are continuously accumulated:

- **CRC error count:** Packets with failed checksums are discarded and counted.
- **Dropout detection:** Gaps in the ESP32 timestamp sequence exceeding 2 ms are counted as dropouts.
- **Jitter statistics:** A rolling window of inter-sample timestamp deltas is maintained to compute mean, max, and standard deviation of sampling jitter.
- **Saturation detection:** Samples at the $\pm$ FSR boundaries (± 32768 raw) are counted per axis, flagging potential clipping events.
- **Noise floor estimation:** Rolling standard deviation of gyroscope magnitude and $|a| - 1$ at rest, providing a live noise-floor proxy.
- **FSYNC hit tracking:** The `fsync_tagged` flag is set when `(temp_raw & 0x01) == 1`, and a cumulative FSYNC hit counter and live rate are maintained.

Each validated sample is passed to a user-registered callback, which the `Session` class uses to feed the `SyncEngine`, `DataLogger`, `OrientationFilter`, and `RepSegmenter`.

6.2.2 CameraReader

The `CameraReader` wraps the `librealsense2` pipeline. It configures the D455 according to Table 5 and starts a background thread that calls `pipeline.wait_for_frames()` in a blocking loop. Each frameset is decomposed into:

- Left and right IR images (`cv::Mat`, `CV_8U`).
- Aligned depth image (`cv::Mat`, `CV_16U`, millimetres).
- Hardware timestamp (via `RS2_FRAME_METADATA_SENSOR_TIMESTAMP`).
- Frame number (monotonic counter).

Camera frames are enqueued into a bounded producer–consumer queue (`CAMERA_QUEUE_LIMIT = 512`). A separate camera worker thread dequeues frames and performs marker tracking, video encoding, and data logging, ensuring that the RealSense streaming thread is never blocked by downstream processing.

When enabled, the D455’s onboard BMI085 IMU streams accelerometer (250 Hz) and gyroscope (200 Hz) data via a separate callback. These samples are logged to `camera_imu.csv` for tripod-shake detection and cross-stream synchronisation validation, but are never used for VBT kinematic computation.

6.3 Session Lifecycle

The `Session` class orchestrates the entire recording workflow through a well-defined state machine:

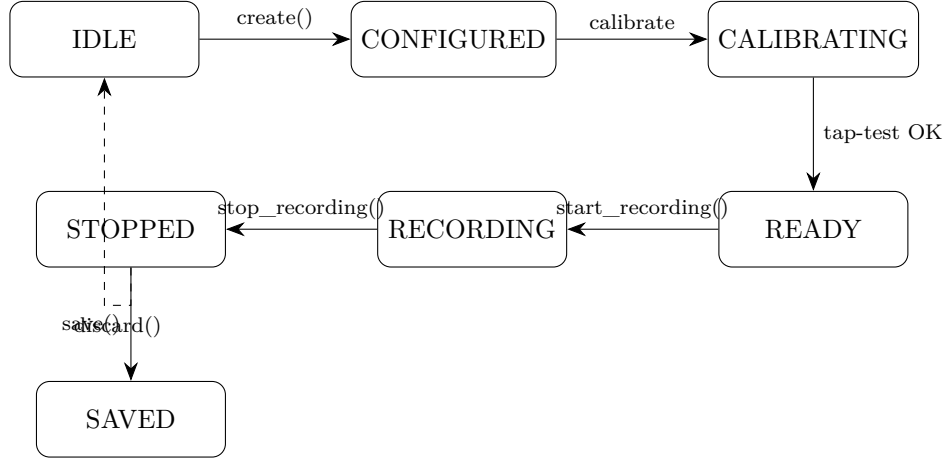


Figure 3: Session state machine.

6.3.1 Atomic Session Finalisation

During recording, all files are written to a directory suffixed with `.partial/`. Upon calling `save()`, the directory is atomically renamed to its final path. If a final directory already exists (e.g., from a previous aborted save), it is moved to a timestamped `.bak_*` backup. This guarantees that a crash or power failure mid-recording never corrupts the dataset—the `.partial` directory remains as a recoverable artefact.

On startup, `Session::find_orphaned_partials()` scans the dataset root for any unfinished `.partial` directories and surfaces them via toast notification, offering the operator the option to recover or discard.

6.3.2 Multi-Set Support

Schema v4 introduces multi-set sessions. During a continuous recording, the operator can call `advance_set(next_set_info)`, which:

1. Stamps `t_end_unified_s` on the current `SetInfo` and records the completed rep count.
2. Appends a new `SetInfo` to the session’s `sets` vector.
3. Instructs the `RepSegmenter` to tag all subsequent reps with the new `set_id`.

6.4 Data Logging

The `DataLogger` writes data in parallel to multiple output streams:

Table 12: Output files generated per session.

Directory	File	Content
imu/	raw_imu.csv	All columns: esp_timestamp_us, host_timestamp_s, unified_time_s, accel_{x,y,z}_g, gyro_{x,y,z}_dps, temp_c, raw integers, fsync_tagged.
imu/	raw_imu.bin	26-byte binary packets (identical to firmware format).
imu/	camera_imu.csv	D455 onboard BMI085 accel/-gyro data.
camera/	marker_positions.csv	Per-frame: timestamp, 2D pixel coords, 3D world coords, blob area, circularity, SNR, confidence, depth source.
camera/	ir_video.mp4	Left IR frames encoded as H.264 video.
camera/	video_frames.csv	Per-frame index: HW timestamp, host timestamp, frame number.
annotations/	rep_segments.json	Per-rep phase boundaries and kinematic summaries.
Root	session_info.json	Complete session metadata (see Section 8).
Root	manifest.json	Per-file SHA-256 checksums and sizes.
Root	events.jsonl	Append-only event log with hash chain.

6.5 Real-Time Processing

6.5.1 Orientation Filter

The real-time GUI uses a Madgwick AHRS filter (`OrientationFilter`) for live gravity-compensated acceleration preview and coordinate-frame visualisation. The filter gain ($\beta = 0.1$) is tuned for responsiveness during live operation. Post-processing in the golden model pipeline replaces this with VQF for superior static accuracy.

6.5.2 Repetition Segmenter

The `RepSegmenter` implements a dual-algorithm approach:

- **Algorithm A (Primary):** Camera-derived velocity zero-crossing detection. A 2nd-order Butterworth lowpass filter (cutoff from the exercise profile, e.g., 10 Hz for squats) is applied to the camera velocity signal. A windowed peak-confirmation detector (`PEAK_WINDOW_N = 20`

samples, ± 0.22 s at 90 fps) identifies confirmed TOP and BOTTOM extrema with prominence thresholds, avoiding false triggers from noise and oscillation.

- **Algorithm B (Fallback):** Raw IMU accelerometer variance detection. When camera tracking is lost for >1 s, the segmenter switches to monitoring the running acceleration variance over a 200 ms window. Motion onset and cessation are detected via variance thresholds.

Each completed repetition is stored as a `RepAnnotation` containing four chronological phase segments: **concentric** $\rightarrow$ **top_rest** $\rightarrow$ **eccentric** $\rightarrow$ **rest**, with associated peak velocity, mean velocity, and ROM.

6.6 Calibration System

The `CalibrationManager` supports:

1. **6-Position Accelerometer Calibration:** The operator places the IMU in six orientations ($\pm X$, $\pm Y$, $\pm Z$ up). For each position, the mean acceleration vector is recorded over ~ 3 s. A least-squares ellipsoid fit computes the 3×3 scale-misalignment matrix **K** and the 3×1 bias vector **b**:

$$\mathbf{a}_{\text{corrected}} = \mathbf{K} (\mathbf{a}_{\text{raw}} - \mathbf{b}) \quad (7)$$

2. **Per-Session Gyroscope Bias:** The first 5 s of stationary data are averaged to compute the gyroscope bias vector, which is subtracted from all subsequent samples.
3. **Camera-to-World Extrinsic:** Gravity alignment from the D455's onboard accelerometer computes the camera-to-world rotation matrix.

All calibration parameters are serialised to JSON with timestamps and sensor serial numbers, forming the `CalibrationProvenance` block in the session metadata.

6.7 Build Provenance

Every build embeds a `Version.h` generated by CMake from `Version.h.in`. This captures:

- Application version (currently v1.1.0).
- Git SHA, branch name, and dirty flag.
- Build timestamp (ISO 8601 UTC).
- Build type (Release/Debug), compiler identification, OS, and architecture.

This fingerprint is embedded into every session's `BuildProvenance` metadata block, ensuring that any kinematic result can be traced back to the exact software state that produced it.

6.8 Graphical User Interface

The GUI is implemented using Dear ImGui with ImPlot for real-time charting. Key panels include:

- **Pre-Flight Panel:** A systematic checklist that gates the START button. Checks include: IMU rate within $\pm 5\%$ of 1 kHz, no accelerometer saturation, camera connected, marker detection rate $> 80\%$, sync drift < 50 ppm, and subject ID populated.
- **Operator View (F12):** A fullscreen, big-numbers display readable from across the gym. Shows: current rep count, last MCV and PCV, velocity-loss percentage, and a stop-set recommendation when VL exceeds the exercise profile threshold.
- **Rep Timeline Panel:** A horizontal NLE-style timeline showing per-rep bars with colour-coded plausibility dots (green = passed, red = failed). Supports drag-handle boundary editing, right-click deletion, and bulk operations.
- **Calibration Wizard (F1):** A step-by-step 6-position guide with live alignment visualisation (dot product + magnitude check), 3-second countdown capture, and a review screen.

- **Replay Mode (F2):** Browse completed sessions, load metadata, view rep annotations, inspect the manifest and event log.

7 Mathematical Data Processing Pipeline

The “Golden Model” pipeline is a multi-stage Python processing chain that transforms raw IMU telemetry into fully validated kinematic trajectories. It is implemented as a sequence of numbered scripts (`step0–step8`) in the `golden_model/` directory, each building upon the outputs of the previous stage. This section presents the mathematical formulation of each stage.

7.1 Step 0: Data Loading and Visualisation

Raw CSV files are loaded and aligned. The IMU time base is derived from the ESP32’s monotonic microsecond counter (`esp_timestamp_us`) rather than the host timestamp, because the host timestamps arrive in bursts of 8 (one per ESP-NOW batch) separated by ~ 8 ms gaps, which would cause numerical instability in `np.gradient` and integration routines. The ESP32 timestamp is anchored to the first IMU sample’s host wall-clock time to share the camera’s temporal reference.

7.2 Step 1: Attitude Estimation and Gravity Removal

The accelerometer measures the vector sum of gravity and the barbell’s true linear acceleration in the *sensor body frame*. To isolate the linear acceleration, we must:

1. Continuously estimate the sensor’s orientation (attitude) relative to the world frame.
2. Rotate the measured acceleration into the world frame.
3. Subtract the known gravity vector.

Orientation is represented as a unit quaternion $\mathbf{q} = [q_w, q_x, q_y, q_z]^T$, avoiding the Gimbal lock singularity inherent to Euler-angle representations.

7.2.1 Rotation via Quaternions

The rotation matrix $\mathbf{R}(\mathbf{q})$ that transforms a vector from the body frame to the world frame is:

$$\mathbf{R}(\mathbf{q}) = \begin{bmatrix} 1 - 2(q_y^2 + q_z^2) & 2(q_x q_y - q_w q_z) & 2(q_x q_z + q_w q_y) \\ 2(q_x q_y + q_w q_z) & 1 - 2(q_x^2 + q_z^2) & 2(q_y q_z - q_w q_x) \\ 2(q_x q_z - q_w q_y) & 2(q_y q_z + q_w q_x) & 1 - 2(q_x^2 + q_y^2) \end{bmatrix} \quad (8)$$

The world-frame linear acceleration is then:

$$\mathbf{a}_w = \mathbf{R}(\mathbf{q}) \mathbf{a}_b - \begin{bmatrix} 0 \\ 0 \\ g \end{bmatrix} \quad (9)$$

where $\mathbf{a}_b$ is the raw accelerometer reading in the body frame and $g = 9.80665 \text{ m/s}^2$.

7.2.2 Initial Orientation from Gravity Vector

The initial quaternion $\mathbf{q}_0$ is computed from the mean acceleration over the first 300 static samples using the TRIAD method:

$$\text{pitch} = -\arcsin(\bar{a}_{x,0}) \quad (10)$$

$$\text{roll} = \arcsin\left(\frac{\bar{a}_{y,0}}{\cos(\text{pitch})}\right) \quad (11)$$

$$\text{yaw} = 0 \quad (\text{undefined without magnetometer}) \quad (12)$$

The Euler angles are converted to a quaternion via the ZYX convention.

7.2.3 Madgwick AHRS Filter

The Madgwick filter [7] fuses gyroscope and accelerometer data using a gradient-descent optimisation that minimises the error between the predicted and measured gravity direction. The quaternion rate of change is:

$$\dot{\mathbf{q}} = \frac{1}{2}\mathbf{q} \otimes \boldsymbol{\omega} - \beta \frac{\nabla f}{\|\nabla f\|} \quad (13)$$

where $\boldsymbol{\omega}$ is the gyroscope angular velocity, f is the objective function measuring the discrepancy between the measured and predicted gravity direction, and β is the filter gain.

Our implementation uses an **adaptive** β : when $|\|\mathbf{a}\| - 1| < 0.1$ (i.e., the measured acceleration is close to $1g$, indicating the sensor is approximately stationary or in uniform motion), $\beta = 0.15$ for aggressive gravity correction; otherwise, $\beta = 0.02$ to avoid corrupting the attitude estimate during high-acceleration dynamic phases.

7.2.4 Multiplicative Extended Kalman Filter (MEKF)

As a cross-validation, the pipeline implements a 6-state MEKF with states $[\delta\boldsymbol{\theta}^T, \delta\mathbf{b}_g^T]^T$ representing the attitude error and gyroscope bias error. The prediction step propagates the nominal quaternion via gyroscope integration:

$$\mathbf{q}_{k+1} = \mathbf{q}_k \otimes \Delta\mathbf{q}(\boldsymbol{\omega} - \mathbf{b}_g, \Delta t) \quad (14)$$

The update step observes the normalised accelerometer vector and corrects the attitude error via a multiplicative quaternion update, with the innovation gated by $|\|\mathbf{a}\| - 1| < 0.2$ to reject dynamic-acceleration phases.

7.2.5 Versatile Quaternion-based Filter (VQF)

The VQF [6] is the most sophisticated attitude estimator in the pipeline. Unlike the Madgwick filter's single-gain structure, VQF implements a three-step architecture:

1. Gyroscope-only quaternion integration.
2. Accelerometer-based inclination correction (independent of angular velocity).
3. Optional magnetometer-based heading correction (not used in our application).

VQF's key advantage for VBT is its separate **rest detection** module: it identifies periods of low motion using configurable thresholds on angular velocity and acceleration variance, enabling faster gyroscope bias convergence during the brief stationary phases between repetitions.

7.3 Step 2: Velocity Integration

The world-frame vertical acceleration $a_{w,z}$ is numerically integrated using the trapezoidal rule:

$$v_z(t_{k+1}) = v_z(t_k) + a_{w,z}(t_k) \cdot \Delta t_k \quad (15)$$

Without drift mitigation, the velocity diverges rapidly. At this stage, simple linear drift removal is applied between manually identified rest boundaries:

$$v_{\text{corr},k} = v_{\text{raw},k} - k \cdot \frac{v_{\text{raw},N}}{N} \quad (16)$$

where N is the segment length and $v_{\text{raw},N}$ is the raw velocity at the segment end.

7.4 Step 3: Autonomous ZUPT Detection

Zero-Velocity Updates (ZUPT) are the primary mechanism for bounding integration drift [10]. The autonomous ZUPT detector identifies stationary periods using a sliding-window variance test:

$$\sigma_{a_z}^2(t) = \text{Var}[a_{w,z}(t - W/2 : t + W/2)] \quad (17)$$

where $W = 200$ ms (200 samples at 1 kHz). A sample is classified as static if $\sigma_{a_z}^2(t) < 0.05 \text{ m}^2/\text{s}^4$. Isolated static detections shorter than 300 ms are suppressed as false positives.

7.5 Step 7: Camera-Oracle ZUPT Strategies

To isolate the bottleneck between ZUPT detection error and orientation drift, the pipeline implements a “camera oracle” experiment that bypasses the IMU stillness detector entirely. Five strategies of ascending fusion are evaluated:

Table 13: Camera-oracle ZUPT integration strategies.

ID	Name	Description
A	Velocity ZUPT only	Force $v = 0$ at camera-detected rest anchors. Linear drift removal between anchors.
B	Velocity + start-position	As A, plus anchor the initial position to <code>cam_pos_i[anchor_0]</code> .
C	Full position + velocity anchor	Force $v = 0$ and $p = p_{\text{cam}}$ at every rest anchor.
D	Dense anchors (every 200 ms)	Pin $v = 0$ and $p = p_{\text{cam}}$ every 200 ms, regardless of motion.
E	Rest-rest + 50 ms position chunks	Outer $v = 0$ boundaries at rest anchors; internal 50 ms position pins to camera.

Camera-oracle rest anchors are located by identifying intervals where $|v_{z,\text{cam}}| < 0.05 \text{ m/s}$ for at least 100 ms. The results of this experiment directly motivated the development of the 15-state ESKF.

7.6 Step 8: 15-State Error-State Kalman Filter

The ESKF is the culmination of the processing pipeline, achieving sub-50 mm position RMSE using only IMU data (no camera fusion) with lifting-specific ZUPT anchors.

7.6.1 State Vector

The nominal state comprises 16 storage elements:

$$\mathbf{x} = [\underbrace{\mathbf{p}^T}_3, \underbrace{\mathbf{v}^T}_3, \underbrace{\mathbf{q}^T}_4, \underbrace{\mathbf{b}_a^T}_3, \underbrace{\mathbf{b}_g^T}_3]^T \quad (18)$$

The error state $\delta\mathbf{x} \in \mathbb{R}^{15}$ uses the minimal rotation parameterisation $\delta\boldsymbol{\theta} \in \mathbb{R}^3$:

$$\delta\mathbf{x} = [\delta\mathbf{p}^T, \delta\mathbf{v}^T, \delta\boldsymbol{\theta}^T, \delta\mathbf{b}_a^T, \delta\mathbf{b}_g^T]^T \quad (19)$$

7.6.2 Process Model

The continuous-time dynamics are:

$$\dot{\mathbf{p}} = \mathbf{v} \quad (20)$$

$$\dot{\mathbf{v}} = \mathbf{R}(\mathbf{q})(\mathbf{a}_m - \mathbf{b}_a) + \mathbf{g}_w \quad (21)$$

$$\dot{\mathbf{q}} = \frac{1}{2} \mathbf{q} \otimes (\boldsymbol{\omega}_m - \mathbf{b}_g) \quad (22)$$

$$\dot{\mathbf{b}}_a = \boldsymbol{\eta}_a \sim \mathcal{N}(\mathbf{0}, \sigma_{ba}^2 \mathbf{I}_3) \quad (23)$$

$$\dot{\mathbf{b}}_g = \boldsymbol{\eta}_g \sim \mathcal{N}(\mathbf{0}, \sigma_{bg}^2 \mathbf{I}_3) \quad (24)$$

where $\mathbf{g}_w = [0, 0, -g]^T$ is the gravity vector in the world frame and $\boldsymbol{\eta}$ represents random walk noise.

7.6.3 Error-State Transition Matrix

The linearised error-state transition matrix $\mathbf{F} = \mathbf{I}_{15} + \Delta t \cdot \mathbf{F}_c$ contains the following active blocks:

$$\mathbf{F}_{pv} = \mathbf{I}_3 \cdot \Delta t \quad (\text{position driven by velocity}) \quad (25)$$

$$\mathbf{F}_{v\theta} = -[\mathbf{R} \mathbf{a}_c]_{\times} \cdot \Delta t \quad (\text{velocity driven by attitude error}) \quad (26)$$

$$\mathbf{F}_{v,ba} = -\mathbf{R} \cdot \Delta t \quad (\text{velocity driven by accel bias}) \quad (27)$$

$$\mathbf{F}_{\theta\theta} = \mathbf{I}_3 - [\boldsymbol{\omega}_c]_{\times} \cdot \Delta t \quad (\text{attitude error propagation}) \quad (28)$$

$$\mathbf{F}_{\theta,bg} = -\mathbf{I}_3 \cdot \Delta t \quad (\text{attitude driven by gyro bias}) \quad (29)$$

where $[\cdot]_{\times}$ denotes the skew-symmetric matrix and $\mathbf{a}_c = \mathbf{a}_m - \mathbf{b}_a$.

7.6.4 ZUPT Observation Model

At rest-flagged samples, we observe $\mathbf{v} = \mathbf{0}$:

$$\mathbf{y}_v = -\mathbf{v}_k, \quad \mathbf{H}_v = [\mathbf{0}_{3 \times 3}, \mathbf{I}_3, \mathbf{0}_{3 \times 9}] \quad (30)$$

The Kalman gain:

$$\mathbf{K} = \mathbf{P} \mathbf{H}_v^T (\mathbf{H}_v \mathbf{P} \mathbf{H}_v^T + \mathbf{R}_v)^{-1} \quad (31)$$

and the error-state correction $\delta\mathbf{x} = \mathbf{K} \mathbf{y}_v$ is injected into the nominal state:

$$\mathbf{p}_k \leftarrow \mathbf{p}_k + \delta\mathbf{p} \quad (32)$$

$$\mathbf{v}_k \leftarrow \mathbf{v}_k + \delta\mathbf{v} \quad (33)$$

$$\mathbf{q}_k \leftarrow \mathbf{q}_k \otimes \delta\mathbf{q}(\delta\boldsymbol{\theta}) \quad (34)$$

$$\mathbf{b}_a \leftarrow \mathbf{b}_a + \delta\mathbf{b}_a \quad (35)$$

$$\mathbf{b}_g \leftarrow \mathbf{b}_g + \delta\mathbf{b}_g \quad (36)$$

The cross-covariance structure of $\mathbf{P}$ is the key mechanism: when velocity is forced to zero, the correction propagates through the $\delta\mathbf{v}\text{--}\delta\boldsymbol{\theta}$, $\delta\mathbf{v}\text{--}\delta\mathbf{b}_a$, and $\delta\mathbf{v}\text{--}\delta\mathbf{b}_g$ cross-terms, actively *debiasing the sensors* as a byproduct of the velocity constraint.

7.6.5 Zero Angular Rate Update (ZARU)

During rest, we additionally observe that the true angular rate should be approximately zero:

$$\mathbf{y}_g = \mathbf{b}_g - \boldsymbol{\omega}_m, \quad \mathbf{H}_g = [\mathbf{0}_{3 \times 12}, -\mathbf{I}_3] \quad (37)$$

This accelerates gyroscope bias convergence during the brief rest phases.

7.6.6 Zero Acceleration Update (ZAU)

At rest, the world-frame linear acceleration should be zero:

$$\mathbf{a}_{\text{pred}} = \mathbf{R}(\mathbf{a}_m - \mathbf{b}_a) + \mathbf{g}_w \stackrel{!}{=} \mathbf{0} \quad (38)$$

Linearising around the current attitude:

$$\mathbf{H}_a = [-\mathbf{R}[\mathbf{a}_c]_{\times}, -\mathbf{R}] \in \mathbb{R}^{3 \times 6} \quad (\text{columns 6--14 of the 15-state}) \quad (39)$$

This observation constrains the pitch and roll components of the attitude error, preventing gyroscope drift from misaligning the gravity subtraction vector.

7.6.7 Lifting-Specific Near-Rest Detector

Unlike pedestrian ZUPT, where foot-strike provides > 300 ms of clean stationary data, a barbell only experiences “rest” for ~ 50 ms at the velocity zero-crossings at the top and bottom of each repetition. VQF’s default rest detector (`restMinT = 1.5 s`) fires only ~ 4 times in a 48-second squat set, starving the ESKF of updates.

Our custom detector accepts windows as short as 50 ms using two simultaneous conditions over a sliding window:

1. $\max_w |a_{w,z}| < 0.5 \text{ m/s}^2$
2. $\max_w |\boldsymbol{\omega}| < 10^\circ/\text{s}$

where w is the window width. The ESKF is designed to tolerate false-positive ZUPT detections gracefully: the observation noise $\mathbf{R}_v$ ensures that a spurious zero-velocity assertion during motion produces a large innovation that is appropriately down-weighted by the Kalman gain.

7.6.8 Filter Noise Parameters

Table 14: ESKF noise parameters.

Parameter	Symbol	Value
Accel measurement noise	σ_a	0.05 m/s^2
Gyro measurement noise	σ_g	0.005 rad/s
Accel bias random walk	σ_{ba}	1×10^{-4}
Gyro bias random walk	σ_{bg}	1×10^{-5}
ZUPT velocity observation	R_v	$(0.001)^2$
ZARU gyro observation	R_g	$(0.005)^2$
ZAU acceleration observation	R_{au}	$(0.05)^2$
Initial $\delta\theta$ uncertainty	$P_{\theta,0}$	$(2^\circ)^2$
Initial $\delta\mathbf{b}_a$ uncertainty	$P_{ba,0}$	$(0.05 \text{ m/s}^2)^2$
Initial $\delta\mathbf{b}_g$ uncertainty	$P_{bg,0}$	$(0.5^\circ/\text{s})^2$

7.7 Repetition Segmentation in the Golden Model

The golden model pipeline segments repetitions by matching confirmed extrema (TOP/BOTTOM) in the ESKF-corrected position trace against the camera ground truth. For each annotated repetition, the pipeline computes:

- Camera ROM vs. IMU ROM (from the max-min position within the rep boundaries).
- Per-rep ROM error in millimetres and percentage.
- Aggregate ROM RMSE and MAE across the set.

8 Dataset Format and Metadata Schema

A research-grade dataset requires not only accurate raw signals but also comprehensive, machine-readable contextual metadata. Without knowing the athlete’s training history, the prescribed load, the subjective fatigue state, and the exact software version that produced the data, no amount of statistical sophistication can produce a reproducible analysis. This section formalises the Schema v4 metadata taxonomy and directory layout.

8.1 Directory Structure

Each session is stored in a self-contained directory. Two layout modes are supported:

```
dataset_root/
  session_20260504_143723/
    imu/
      raw_imu.csv
      raw_imu.bin
      camera_imu.csv
    camera/
      marker_positions.csv
      ir_video.mp4
      video_frames.csv
    annotations/
```

```

    rep_segments.json
calibration/
    accel_calib.json
    gyro_bias.json
    camera_extrinsic.json
session_info.json
manifest.json
events.jsonl

```

BIDS Mode (Multi-Subject Archiving): When enabled, the directory follows the Brain Imaging Data Structure (BIDS) convention adapted for movement science:

```

dataset_root/
  sub-A7F3/
    ses-2026-05-04/
      run-01_squat/
        imu/ camera/ annotations/ calibration/
        session_info.json
        manifest.json
        events.jsonl

```

The subject identifier is the SHA-256 hash of the participant ID, enabling dataset publication with anonymised file paths while maintaining internal consistency for longitudinal studies.

8.2 Schema Versioning

Every `session_info.json` and `vbt_config.json` carries a top-level `schema_version` integer. The current version is 4. The loading logic enforces the following invariant:

- Configs with `schema_version > CURRENT_SCHEMA` are refused with a clear error message, preventing silent misinterpretation of newer-format fields.
- Configs with `schema_version <= CURRENT_SCHEMA` are loaded using `_WITH_DEFAULT_` macros, which populate missing fields with struct defaults. This allows sessions recorded under Schema v2 to be loaded and re-exported by Schema v4 software without data loss.

8.3 Session Metadata Taxonomy: `SessionInfo`

The `SessionInfo` struct contains over 120 fields, organised into nine logical groups. Table 15 provides an overview; subsequent subsections detail the non-trivial groups.

Table 15: Overview of `SessionInfo` metadata groups.

Group	Fields	Primary Content
Identity	6	Session ID, timestamp, schema version, format.
Subject	14	Anonymised hash, sex, age, mass, height, training experience.
Exercise	8	Exercise name, variant, depth criterion, tempo, exercise profile.
Training Context	13	Prescription (%1RM, target reps/sets), mesocycle phase, RIR.
Conditions	6	Time of day, freshness rating, warmup status, temperature, humidity.
Equipment	10	Bar mass/make, plate manufacturer, rack ID, IMU mount details.
Calibration Prov.	8	Calibration file paths, timestamps, gyro biases, sync residual stats.
Build Provenance	8	Software version, git SHA, branch, dirty flag, build timestamp.
Operator	4	Operator ID, notes, preflight override audit trail.
Sets	$n \times 11$	Per-set: weight, target reps, actual reps, timing, RPE, quality.

8.4 Subject Metadata: `SubjectInfo`

Subject data is stored in a separate file indexed by the anonymised subject hash, enabling the separation of personally identifiable information (PII) from session data for ethical data management.

Table 16: `SubjectInfo` fields (stored in `sub_registry/`).

Field	Type	Description
<code>subject_hash</code>	string	SHA-256 of the participant identifier.
<code>sex</code>	string	"male", "female", or "other".
<code>age_years</code>	int	At the time of data collection.
<code>body_mass_kg</code>	float	Session-day weigh-in.
<code>height_cm</code>	float	Measured or self-reported.
<code>training_exp_years</code>	float	Total years of resistance training experience.
<code>dominant_side</code>	string	"left", "right", or "ambidextrous".
<code>arm_length_cm</code>	float	Required for press exercises (determines lockout height).
<code>thigh_length_cm</code>	float	Required for squat depth estimation from bar position.
<code>tibia_length_cm</code>	float	Affects squat kinematics (ankle mobility compensation).
<code>recent_1rm</code>	map<string,float>	Exercise $\rightarrow$ 1RM. E.g., {"back_squat": 180.0}.
<code>injury_notes</code>	string	Free-form current injury/limitation text.
<code>consent_version</code>	string	Version identifier of the signed informed consent form.
<code>ethics_protocol</code>	string	Institutional ethics protocol number.
<code>data_sharing_tier</code>	string	"none", "anonymised", or "full".

8.5 Training Context: `SetInfo`

Each session may contain one or more working sets. The `SetInfo` struct captures:

Table 17: SetInfo fields (per set within a session).

Field	Type	Description
set_id	int	1-indexed within the session.
weight_kg	float	Total barbell mass (bar + plates).
target_reps	int	Programmed repetitions for this set.
target_rpe	float	Programmed Rating of Perceived Exertion (6–10 Borg CR10).
actual_rpe	float	Operator-reported RPE after completion.
velocity_stop	string	"none", "v120" (20% VL cutoff), "absolute_0.3" (cut at < 0.3 m/s).
completed_reps	int	Actual reps detected (from segmenter).
t_start_unified_s	double	Wall-clock start time.
t_end_unified_s	double	Wall-clock end time.
quality	string	"good", "suspect", "failed", "discarded".
notes	string	Free-form operator notes.

8.6 Equipment Provenance: EquipmentInfo

Following the recommendation of Weakley et al. [11] that IMU placement significantly affects measurement accuracy, the equipment block documents:

Table 18: EquipmentInfo fields.

Field	Type	Description
bar_make	string	Manufacturer and model.
bar_mass_kg	float	Nominal mass (e.g., 20.0).
plate_manufacturer	string	Plate type affects drop dynamics.
rack_id	string	Station identifier for multi-rack facilities.
imu_mount_location	string	"left_collar", "right_collar", "sleeve_end", "center_knurl".
imu_mount_orientation	string	"z_up", "y_up", etc. Determines the gravity alignment axis.
camera_distance_m	float	Camera-to-barbell distance. Affects depth noise ($\propto z^2$).
camera_mount_type	string	"tripod", "wall_mount", "floor".
lighting_condition	string	"natural", "artificial", "mixed".
marker_type	string	"retroreflective_tape", "retroreflective_ball".

8.7 Calibration Provenance

To ensure traceability from kinematic output to calibration state, each session records:

- Absolute path and timestamp of the accelerometer calibration file (`accel_calib.json`).
- Absolute path and timestamp of the camera extrinsic calibration.
- The specific gyroscope bias vector used ($[\delta\omega_x, \delta\omega_y, \delta\omega_z]$, in $^\circ/\text{s}$).
- Tap-test sync residual: offset (ts), drift (ppm), and correlation quality.
- Placeholder for an Allan-variance characterisation file (for future IEEE 952-2020 compliance).

8.8 Event Log (`events.jsonl`)

Every session generates an append-only, newline-delimited JSON event log. Each event carries a timestamp, event type, and free-form data payload. The event types include:

- | | |
|-----------------------------------|--------------------------------------|
| • <code>session_created</code> | • <code>sync_rearm_triggered</code> |
| • <code>session_started</code> | • <code>preflight_override</code> |
| • <code>session_stopped</code> | • <code>calibration_completed</code> |
| • <code>session_saved</code> | • <code>rep_manual_mark</code> |
| • <code>set_advanced</code> | • <code>rep_deleted</code> |
| • <code>tap_test_completed</code> | • <code>operator_note</code> |

The `manifest.json` file includes a SHA-256 hash of the entire `events.jsonl`, providing tamper detection for the audit trail.

8.9 Anonymisation Pipeline

The `scripts/anonymise_session.py` script produces a publication-ready copy of any session by:

1. Replacing `subject_id` with its SHA-256 hash.
2. Redacting operator notes, operator ID, and location fields.
3. Stripping operator-initiated events from `events.jsonl`.
4. Refusing to overwrite an existing output directory (to prevent accidental re-anonymisation).

The anonymised session retains all kinematic data, timing information, and non-PII metadata, making it suitable for deposit on Zenodo or similar open-data repositories.

9 Validation and Quality Assurance

A research-grade data collection platform must provide verifiable evidence that every stage of the measurement chain—from the ADC conversion to the final kinematic trajectory—meets quantitative accuracy targets. This section describes the multi-tiered validation framework, comprising pre-session hardware verification, in-session real-time monitoring, and post-session signal-processing validation.

9.1 Pre-Session Validation: Pre-Flight Checklist

Before any recording can begin, the GUI enforces a systematic pre-flight checklist (`PreflightPanel`). The START button is gated; only when all checks pass (or the operator explicitly overrides with documented justification) does the system transition to the RECORDING state.

Table 19: Pre-flight checklist items and acceptance criteria.

Check	Pass	Warn	Fail
IMU sample rate	Within $\pm 5\%$ of 1 kHz (950–1050 Hz)	Within $\pm 10\%$	Outside $\pm 10\%$ or no samples
Accel saturation	0 clipped samples	< 10 clipped	≥ 10 clipped
Camera connected	<code>is_open() == true</code>	—	Not connected
Marker detection	Detection rate $> 80\%$	$> 50\%$	$\leq 50\%$
Marker SNR	> 3.0	> 2.0	≤ 2.0
Sync drift	< 50 ppm	< 100 ppm	≥ 100 ppm or rearm flagged
Subject ID	Non-empty	—	Empty

Any override of a failed check is recorded as a `preflight_override` event in `events.jsonl`, including the specific check that was overridden and the operator’s reason, preserving the audit trail for downstream data quality review.

9.2 In-Session Monitoring

9.2.1 IMU Signal Quality Indicators

The following metrics are computed in real-time and displayed in the Sensor panel:

- **Measured rate:** Rolling estimate of the actual IMU sample rate from ESP32 timestamps. Deviations $> 5\%$ from nominal trigger a toast warning.
- **Jitter statistics:** Mean, max, and standard deviation of inter-sample timing jitter. Jitter $> 100\text{ }\mu\text{s}$ indicates potential SPI bus contention or RTOS priority inversion.
- **Per-axis saturation count:** Cumulative count of samples at ± 32768 (full-scale ADC). Persistent saturation indicates an insufficient FSR.
- **Gyro noise floor:** Rolling standard deviation of $|\omega|$ during detected rest periods. Expected $< 0.1^\circ/\text{s}$ for the ICM-42688-P.
- **Accel noise floor:** Rolling standard deviation of $|\mathbf{a}| - 1$ at rest. Expected $< 0.005\text{ g}$.
- **CRC error rate:** Cumulative count and rate of packets failing the CRC16-CCITT check.
- **Dropout count:** Number of gaps in the ESP32 timestamp sequence exceeding 2 ms.

9.2.2 FSYNC Health Monitoring

The FSYNC hit rate is tracked in real-time and should match the camera frame rate ($\sim 90\text{ Hz}$). A mismatch indicates a fault in the level-shifting circuit, a loose SYNC cable, or a camera configuration error.

9.2.3 Synchronisation Drift Monitoring

The `SyncEngine` continuously computes the clock drift between the ESP32 and D455 hardware clocks using the FSYNC anchor pairs (Section 4.6). If drift exceeds 100 ppm for 5 s, the auto-rearm mechanism is triggered, halting data collection until the operator performs a fresh tap test.

9.2.4 Per-Repetition Kinematic Plausibility

After each repetition is segmented, the **Validator** applies configurable plausibility bounds:

Table 20: Per-repetition kinematic plausibility bounds (configurable per exercise profile).

Metric	Unit	Min	Max
Mean Concentric Velocity	m/s	0.05	3.00
Peak Concentric Velocity	m/s	0.10	5.00
Range of Motion	m	0.05	1.50
Concentric Duration	s	0.10	10.00

Repetitions that violate any bound are flagged as implausible. The Rep Timeline Panel displays a colour-coded dot for each rep: green for passed, red for failed. Hovering over a red dot shows the specific violations (e.g., “PCV exceeds 5.00 m/s: likely double-counted bounce”).

9.3 Post-Session Validation: Golden Model Pipeline

9.3.1 Camera–IMU Position Agreement

The primary validation metric is the Root Mean Square Error (RMSE) between the ESKF-derived position trajectory and the camera ground truth, computed over the active zones of each session:

$$\text{RMSE}_p = \sqrt{\frac{1}{N_a} \sum_{k \in \text{active}} (p_{z,\text{ESKF}}(t_k) - p_{z,\text{cam}}(t_k))^2} \quad (40)$$

The target is **RMSE** < 50 mm (5 cm).

9.3.2 Velocity Agreement

Velocity RMSE is computed similarly:

$$\text{RMSE}_v = \sqrt{\frac{1}{N_a} \sum_{k \in \text{active}} (v_{z,\text{ESKF}}(t_k) - v_{z,\text{cam}}(t_k))^2} \quad (41)$$

Per-rep peak velocity errors are compared against the Courel-Ibáñez equivalence margin of ± 0.07 m/s.

9.3.3 Bland–Altman Analysis

For each paired measurement (camera vs. ESKF), the Bland–Altman method [4] is applied:

$$\bar{d} = \frac{1}{N} \sum_{i=1}^N (x_{\text{ESKF},i} - x_{\text{cam},i}) \quad (\text{mean bias}) \quad (42)$$

$$\text{LoA} = \bar{d} \pm 1.96 \cdot s_d \quad (95\% \text{ Limits of Agreement}) \quad (43)$$

where s_d is the standard deviation of the differences. The **Validator** class computes and persists these metrics in its JSON report.

9.3.4 Per-Rep ROM Accuracy

For each annotated repetition, the golden model computes:

$$\text{ROM}_{\text{cam}} = \max_{t \in \text{rep}} p_{\text{cam}}(t) - \min_{t \in \text{rep}} p_{\text{cam}}(t) \quad (44)$$

$$\text{ROM}_{\text{ESKF}} = \max_{t \in \text{rep}} p_{\text{ESKF}}(t) - \min_{t \in \text{rep}} p_{\text{ESKF}}(t) \quad (45)$$

and reports the per-rep error, aggregate ROM RMSE, and ROM MAE.

9.4 Synchronisation Validation: Tap Test

The tap test (Section 4.6) serves as both calibration and validation. After the test, the system reports:

- **Temporal offset:** The systematic IMU-to-camera delay. Expected < 1 ms with the hardware FSYNC path.
- **Drift rate:** The rate of clock divergence. Expected < 50 ppm (0.18 s/h).
- **Cross-correlation quality:** The normalised cross-correlation coefficient between the IMU acceleration magnitude and the camera position displacement. Expected > 0.9 .

Multiple tap tests within a session can be used to verify that drift remains bounded throughout the recording duration.

9.5 Allan Variance Characterisation

For IMU noise characterisation per IEEE 952-2020 [1], the system includes a planned Allan variance analysis pipeline:

- A static recording of 4–12 hours is collected with the IMU motionless on a flat surface.
- The Allan deviation $\sigma(\tau)$ is computed as a function of averaging time τ for each accelerometer and gyroscope axis.
- The log-log plot of $\sigma(\tau)$ vs. τ reveals:
 - **Angle/Velocity Random Walk (ARW/VRW):** Slope = $-1/2$ region.
 - **Bias Instability:** Minimum of the curve.
 - **Rate Random Walk:** Slope = $+1/2$ region.
- These parameters feed directly into the ESKF process-noise covariance matrix $\mathbf{Q}$.

9.6 Diagnostic Export

At any time during or after a session, the operator can trigger a diagnostic bundle export (F5 or Tools → Export Diagnostics). This produces a PII-free JSON file containing:

- Build provenance (version, git SHA, compiler).
- Current IMU/camera/sync statistics.
- Recent toast notification history.
- Full application configuration (with PII fields redacted).

This bundle enables remote troubleshooting without requiring access to the raw session data.

9.7 Validation Summary and Acceptance Criteria

Table 21: System-level validation targets.

Metric	Target	Measurement Method
Position RMSE (ESKF vs. camera)	< 50 mm	Golden model Step 8
Velocity bias	< 0.05 m/s	Bland–Altman mean difference
Velocity LoA (95%)	$< \pm 0.10$ m/s	Bland–Altman 1.96σ
Synchronisation offset	< 1 ms	Tap test
Clock drift	< 50 ppm	FSYNC anchor pairs
IMU sample rate stability	$\pm 5\%$ of 1 kHz	ESP32 timestamp analysis
CRC error rate	$< 0.01\%$	Packet validation
Marker detection rate	$> 80\%$	Session statistics

Acknowledgements

The author would like to thank ...

References

- [1] IEEE standard specification format guide and test procedure for single-axis interferometric fiber optic gyros, 2020.
- [2] Carlos Balsalobre-Fernández, David Marchante, Eneko Baz-Valle, Iván Alonso-Molero, Sergio L. Jiménez, and Mario Muñoz-López. Analysis of wearable and smartphone-based technologies for the measurement of barbell velocity in different resistance training exercises. *Frontiers in Physiology*, 8:649, 2017.
- [3] Harry G. Banyard, Kazunori Nosaka, and G. Gregory Haff. Reliability and validity of the load-velocity relationship to predict the 1rm back squat. *Journal of Strength and Conditioning Research*, 31(7):1897–1904, 2017.
- [4] J. Martin Bland and Douglas G. Altman. Statistical methods for assessing agreement between two methods of clinical measurement. *The Lancet*, 327(8476):307–310, 1986.
- [5] Juan José González-Badillo and Luis Sánchez-Medina. Movement velocity as a measure of loading intensity in resistance training. *International Journal of Sports Medicine*, 31(5):347–352, 2010.
- [6] Daniel Laidig and Thomas Seel. VQF: Highly accurate IMU orientation estimation with bias estimation and magnetic disturbance rejection. *Information Fusion*, 91:187–204, 2023.
- [7] Sebastian O. H. Madgwick, Andrew J. L. Harrison, and Ravi Vaidyanathan. Estimation of IMU and MARG orientation using a gradient descent algorithm. *2011 IEEE International Conference on Rehabilitation Robotics*, pages 1–7, 2011.
- [8] Michael R. McGuigan and Barry D. Wilson. Biomechanical analysis of the deadlift. *Journal of Strength and Conditioning Research*, 10(4):250–255, 2004.
- [9] Alejandro Pérez-Castilla, Alessandro Piepoli, Gonzalo Delgado-García, Germán Garrido-Blanca, and Amador García-Ramos. Reliability and concurrent validity of seven commercially available devices for the assessment of movement velocity at different intensities during the bench press. *Journal of Strength and Conditioning Research*, 33(5):1258–1265, 2019.
- [10] Isaac Skog, Peter Händel, John-Olof Nilsson, and Jouni Rantakokko. Zero-velocity detection—an algorithm evaluation. *IEEE Transactions on Biomedical Engineering*, 57(11):2657–2666, 2010.
- [11] Jonathon Weakley, Bryan Mann, Harry Banyard, Shaun McLaren, Tannath Scott, and Amador Garcia-Ramos. Velocity-based training: From theory to application. *Sports Medicine*, 51(6):1257–1270, 2021.